

A FIELD GUIDE TO REAL ENGINEERING · PART ONE

Foundations, Data & Auth

Building the Student Expense Tracker from an empty scaffold to a working, tested, database-backed API with real authentication — and everything that broke along the way.

SERIES

Part 1 of 3

STACK

FastAPI · SQLAlchemy ·
PostgreSQL · React

READER

Knows Python & JS syntax,
new to backend
engineering

Student Expense Tracker · Team 17

Foundations · Database · Migrations · Auth

01

YOU ARE HERE

Foundations, Data & Auth

Project anatomy, the database layer, migrations, configuration, request validation, and real JWT authentication. By the end, a user can register, log in, and create expenses through a protected API — the load-bearing wall everything else stands on.

02

Building the Product Surface

Turning the API into something a user actually touches: the expense form and list, budgets with overspend warnings, and the state-management patterns that keep a React app honest about what the server knows.

03

Analytics & the Finished Product

Aggregation queries, the charting layer, and turning a working CRUD app into something that actually answers the question “where did my money go?”

CONTENTS

01 **How the Web Actually Works Here**

Client, server, database — the three-tier model this whole project is an instance of.

02 **Anatomy of a Real Backend Project**

Why the code is split into models, schemas, routers, and utils — and what FastAPI is doing at startup.

03 **The Database Layer**

Tables, keys, relationships, and how SQLAlchemy turns rows into Python objects.

04 **Migrations with Alembic**

Why you never hand-edit a production schema, and how autogenerate actually works.

05 **Configuration & Secrets**

Keeping passwords and connection strings out of your source code, properly.

06 **Data Validation with Pydantic**

Why the shape of an API request is never the shape of a database row.

07 **Authentication & Security, For Real**

Hashing, JSON Web Tokens, and FastAPI's dependency-injection convention.

08 **Building Real CRUD Endpoints**

REST conventions, and the difference between authentication and authorization.

09 **War Stories: Debugging Like an Engineer**

Three real bugs from this build, read the way a working engineer reads them.

10 **Frontend Foundations**

The thin layer of React that turns a JWT into a logged-in user experience.

11 **Proving It Works**

Testing the whole flow by hand before trusting it.

12 **Recap & What's Next**

Every concept introduced, in one table, and a look ahead to Part 2.

13 **Glossary**

Every term this guide introduced, defined in one line.

How the Web Actually Works Here

Before touching a single file, it's worth being precise about what "backend" and "frontend" actually mean — because every decision in this guide follows from this one picture.

A student opens the Expense Tracker, types in a coffee they bought, and hits save. Nothing about that sentence tells you where the number "4.50" actually goes. It has to travel from a browser tab, across a network, into a program that decides whether it's allowed to be saved, into a database that remembers it forever, and back again — all in well under a second. This project, like almost every web application you'll ever build or use, is three separate programs pretending to be one.

The **frontend** is the part that runs on the user's machine, inside their browser: the React application. It draws buttons, reacts to clicks, and knows nothing about databases. The **backend** is a separate program — our FastAPI application — that runs on a server, has no idea what a button looks like, and exists purely to receive requests, apply business rules, and talk to the database. The **database** is a third program again, PostgreSQL, whose entire job is storing data durably and answering questions about it quickly.

These three programs never share memory. They only ever talk to each other by sending messages over a network — even when, as in local development, that "network" is just `localhost` talking to itself. That constraint is not an inconvenience; it's the whole point. It's what lets you deploy the database on one machine, the API on another, and serve the frontend from a CDN on a third, and it's what lets three teammates work on three different layers without stepping on each other's code.

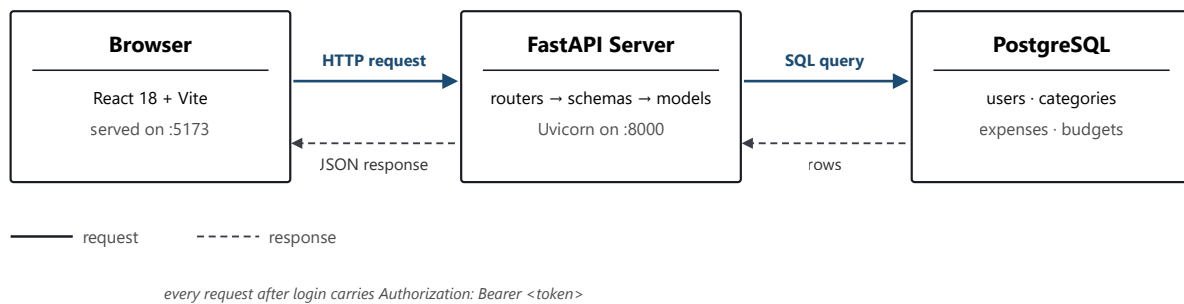


Fig. 1 — The three tiers this project is built from. Every hop the browser makes into FastAPI is HTTP; every hop FastAPI makes into Postgres is SQL. Two different protocols, two different trust boundaries, and (as Chapter 7 covers) two different places authentication has to be checked.

Two more ideas set up everything that follows. First, this is a **REST API**: the backend exposes a set of URLs (`/expenses` , `/auth/login`) that respond to standard HTTP verbs — `GET` to read, `POST` to create, `PUT` to update, `DELETE` to remove — and communicates in JSON. There's no custom protocol to learn; if you understand a URL and a JSON object, you already understand the shape of every conversation this app has with itself.

Second, the backend is **stateless**. FastAPI doesn't remember who you are between requests — there's no "logged in session" sitting in server memory. Every single request has to prove who's making it, from scratch, using the JWT covered in Chapter 7. This feels like more work up front, but it's what lets you run five copies of the API behind a load balancer with zero coordination between them. Statelessness is a constraint you'll feel throughout this guide, and it's worth understanding as a deliberate choice, not an omission.

💡 CONCEPT

Why not just use one language for everything? You could write the backend in JavaScript too (Node.js does this well). This project uses Python for the backend because the concepts — ORMs, migrations, dependency injection, typed request validation — are the industry-standard way backend engineering is taught and practiced across languages. Learn them in Python here and you'll recognize the same shapes in Django, Rails, Spring, or a Node/Nest API later.

Anatomy of a Real Backend Project

The folder structure isn't bureaucracy. Each directory is a promise about what kind of code lives there — and that promise is what lets a project survive past the first afternoon.

Open `backend/app/` and you'll find five directories before you find a single line of business logic. That's deliberate, and it's the first real engineering convention this project teaches: **separation of concerns**. Every file answers exactly one kind of question, and nothing else is allowed to leak in.

```
backend/app/  
├ models/      # the shape of the data, as PostgreSQL tables (SQLAlchemy)  
├ schemas/     # the shape of the data, as API requests/responses (Pydantic)  
├ routers/     # the URLs: what happens when a request arrives  
├ utils/       # password hashing, JWTs, category seeding - reusable logic  
├ database.py  # the one place that knows how to talk to Postgres  
├ config.py    # settings, loaded from environment variables  
├ dependencies.py # shared request-time logic (who is the current user?)  
└ main.py      # wires everything above into one running application
```

Notice what's *not* here: there's no single 2,000-line `app.py` with everything jammed together. A model file never imports a router. A router never talks to the database directly — it always goes through a session that `database.py` handed it. This isn't a rule for its own sake: it means that six months from now, "why is this expense showing the wrong amount" is a question you can answer by reading *one* file (`routers/expenses.py`), not by re-reading the entire application.

Everything converges in `main.py`, which does four jobs and only four jobs: create the FastAPI application, attach middleware, register a startup task, and mount each router under a URL prefix.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app import models  # noqa: F401 - ensures all models are registered before mappers
configure
from app.config import settings
from app.database import SessionLocal
from app.routers import analytics, auth, budgets, categories, expenses
from app.utils.seed import seed_default_categories

app = FastAPI(title="Expense Tracker API", version="0.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins.split(","),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.on_event("startup")
def on_startup() → None:
    db = SessionLocal()
    try:
        seed_default_categories(db)
    finally:
        db.close()

app.include_router(auth.router, prefix="/auth", tags=["auth"])
app.include_router(expenses.router, prefix="/expenses", tags=["expenses"])
app.include_router(categories.router, prefix="/categories", tags=["categories"])
app.include_router(budgets.router, prefix="/budgets", tags=["budgets"])
app.include_router(analytics.router, prefix="/analytics", tags=["analytics"])

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

Two details are worth slowing down on. First, `CORSMiddleware`. By default, a browser refuses to let JavaScript running on one origin (`http://localhost:5173`) make requests to a different origin (`http://localhost:8000`) — this is the browser's **Cross-Origin Resource Sharing** policy, and it exists to stop a malicious site from silently calling your bank's API using your logged-in cookies. Because our frontend and backend genuinely are different origins, we have to explicitly tell FastAPI which origins are allowed to call it. That list comes from `settings.cors_origins` — configuration, not a hardcoded string, which is the whole subject of Chapter 5.

Second, the `@app.on_event("startup")` function. FastAPI lets you register code that runs exactly once, when the server process boots, before it accepts any requests. We use it to guarantee the six default expense categories (Food, Transport, Books...) always exist, without a human ever having to run a seed script by hand. It's a small thing, but it's the difference between "works on my machine because I remembered to seed it" and "works, period."



UNDER THE HOOD

That `from app import models` line at the top looks unused — nothing in `main.py` references `models` directly, and a linter will flag it (hence the `noqa`). It's there for a genuinely subtle reason involving how SQLAlchemy resolves relationships between tables at startup, and it's exactly the kind of thing that will 500 your app if you remove it. We hit this for real; the full story, traceback and all, is in Chapter 9.

The Database Layer

Every expense this app will ever store starts as a Python class. Understanding why that's true — and what it costs — is the single most important idea in backend engineering.

A relational database like PostgreSQL organizes data into **tables**: grids of rows and columns, similar to a spreadsheet but with rules a spreadsheet doesn't enforce. Every row has a **primary key** (`id`) that uniquely identifies it, and rows in one table can point at rows in another table using a **foreign key** — a column that stores another table's primary key. That single mechanism, a number in one table referring to a row in another, is how a database represents relationships: an expense "belongs to" a user because its `user_id` column holds that user's `id`.

This project has four tables, and the relationships between them tell you almost everything about how the app behaves before you read a single endpoint:

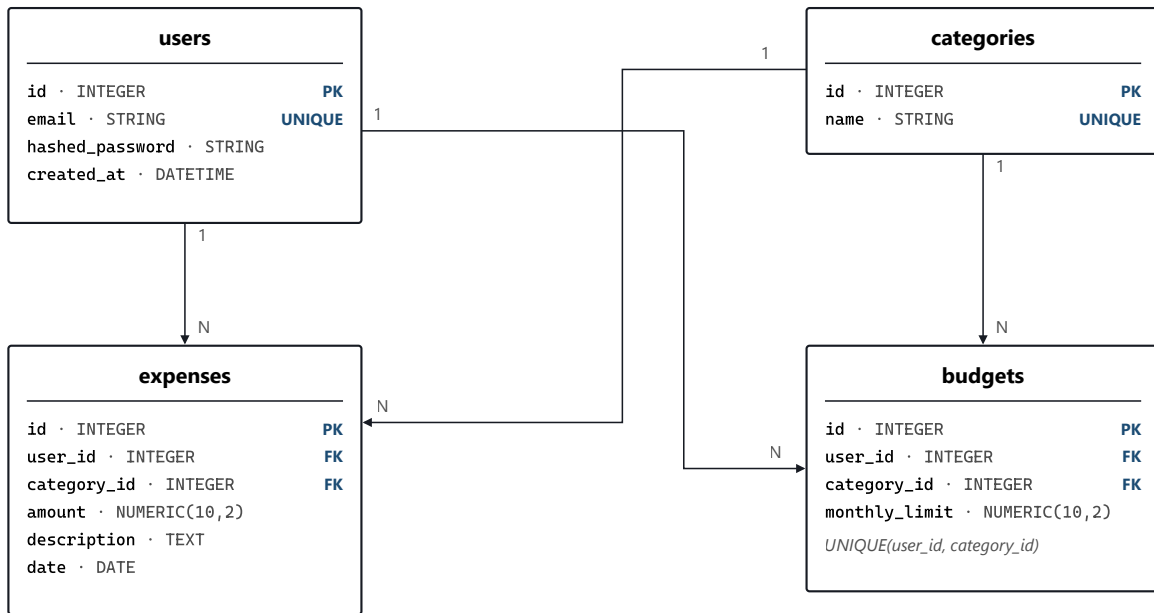


Fig. 2 — Every expense and every budget belongs to exactly one user and exactly one category (the “N” end), while a single user or category can have many expenses and budgets (the “1” end) — the one-to-many relationship that shows up constantly in real schemas.

Why an ORM instead of writing SQL by hand

You could talk to Postgres by writing raw SQL strings: `SELECT * FROM expenses WHERE user_id = 4`. It would work. It also becomes a liability fast — string-built SQL is how SQL-injection vulnerabilities happen, there's no help from your editor, and every table change means hunting down every string that mentions it. An **ORM** (Object-Relational Mapper) like **SQLAlchemy** solves this by letting you describe a table as a Python class once, and then work with rows as ordinary Python objects — `expense.amount` instead of parsing a tuple, `db.query(Expense).filter(...)` instead of string concatenation. The trade you're making is a small amount of "magic" for a large amount of safety and editor support, and it's the standard trade in professional backend work.

Every model in this project inherits from a single `Base` class, declared once in `database.py`:

backend/app/database.py

PYTHON

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from app.config import settings

engine = create_engine(settings.database_url)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

The `engine` is the connection pool that actually knows how to reach Postgres. `SessionLocal` is a factory for **sessions** — think of a session as a single conversation with the database, tracking every object you've loaded or changed so it can turn them into the right SQL when you call `commit()`. And `get_db()` is a **generator function** — note the `yield` instead of `return` — that FastAPI will call before every request that needs one, guaranteeing the session is always closed in the `finally` block even if the request handler raises an exception. You'll see this exact function reappear as a *dependency* in nearly every router in Chapter 8.

The four models

`User` is the simplest and the root of everything else. Two relationships are declared on it — `expenses` and `budgets` — using SQLAlchemy's `relationship()`, which is different from a foreign key column: it doesn't create a database column at all. It's a purely Python-side convenience that lets you write `user.expenses` and get a list of that user's `Expense` objects back, with SQLAlchemy generating the right `JOIN` or lazy-loaded query behind the scenes.

backend/app/models/user.py

PYTHON

```
from sqlalchemy import Column, DateTime, Integer, String, func
from sqlalchemy.orm import relationship

from app.database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, index=True, nullable=False)
    hashed_password = Column(String, nullable=False)
    created_at = Column(DateTime(timezone=True), server_default=func.now(), nullable=False)

    expenses = relationship("Expense", back_populates="user", cascade="all, delete-orphan")
    budgets = relationship("Budget", back_populates="user", cascade="all, delete-orphan")
```

Three details worth naming: `unique=True` on `email` is a database-level constraint — Postgres itself will refuse a second row with a duplicate email, even if application code has a bug that forgets to check. `index=True` tells Postgres to maintain a fast lookup structure on that column, because "find the user by email" (every login) is a query we'll run constantly. And `cascade="all, delete-orphan"` answers a question you'd otherwise discover the hard way in production: if a user is deleted, their expenses and budgets are deleted with them, instead of becoming orphaned rows pointing at a `user_id` that no longer exists.

`Expense` is where the foreign keys actually appear as real columns:

```
from sqlalchemy import Column, Date, ForeignKey, Integer, Numeric, String, Text
from sqlalchemy.orm import relationship

from app.database import Base

class Expense(Base):
    __tablename__ = "expenses"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False, index=True)
    category_id = Column(Integer, ForeignKey("categories.id"), nullable=False, index=True)
    amount = Column(Numeric(10, 2), nullable=False)
    description = Column(Text)
    date = Column(Date, nullable=False, index=True)

    user = relationship("User", back_populates="expenses")
    category = relationship("Category", back_populates="expenses")
```



CONCEPT

Why `Numeric(10, 2)` and not a float? Floating-point numbers can't represent most decimal fractions exactly — `0.1 + 0.2` famously doesn't equal `0.3` in almost every programming language. For money, that rounding error compounds into real discrepancies. `Numeric(10, 2)` tells Postgres to store an exact decimal with up to 10 total digits and 2 after the point — exactly how currency is meant to behave. This is a rule with no exceptions in financial software: never store money as a float.

Notice the two relationship names, `user` and `category`, are the mirror image of the ones declared on `User`. `back_populates` is what wires them together: it tells SQLAlchemy that `Expense.user` and `User.expenses` are two sides of the *same* relationship, so that appending to one automatically keeps the other consistent in memory. Get the string wrong on either side and SQLAlchemy will fail loudly at startup rather than silently misbehaving — a deliberate design choice on their part that we'll be grateful for in Chapter 9.

`Budget` looks almost identical to `Expense`, but adds something new: a constraint that spans *two* columns together.

```
from sqlalchemy import Column, ForeignKey, Integer, Numeric, UniqueConstraint
from sqlalchemy.orm import relationship

from app.database import Base

class Budget(Base):
    __tablename__ = "budgets"
    __table_args__ = (UniqueConstraint("user_id", "category_id",
name="uq_budget_user_category"),)

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False, index=True)
    category_id = Column(Integer, ForeignKey("categories.id"), nullable=False, index=True)
    monthly_limit = Column(Numeric(10, 2), nullable=False)

    user = relationship("User", back_populates="budgets")
    category = relationship("Category", back_populates="budgets")
```

`UniqueConstraint("user_id", "category_id")` says: a single user may have *one* budget for "Food," not two competing ones. Neither column is unique on its own — plenty of rows can share a `user_id`, plenty can share a `category_id` — but the *pair* must be unique. This is the database enforcing a business rule that would otherwise live only in application code (and would eventually be bypassed by some code path that forgot to check it).

⚠️ COMMON MISTAKE

A tempting shortcut is to skip constraints like this and "just check in Python before inserting." The problem is concurrency: two requests can both check "does a budget exist?", both see no, and both insert — a classic race condition. A database constraint is enforced atomically by Postgres itself, so it's the only place this rule is actually safe.

Migrations with Alembic

The models in Chapter 3 describe the database we want. They do not create it. That gap — and how to cross it safely, forever, without ever losing data — is what migrations are for.

Imagine the naive approach: you write your `Expense` model, and on every app startup you tell SQLAlchemy to "just make sure the tables match the models." That works beautifully — right up until this app has real users with real data, and you need to rename a column. There's no way for an automated "make it match" process to know whether renaming `amount` to `amount_cents` means *rename this column* or *delete this column and add an unrelated one*. One preserves every user's data; the other destroys it. A production database schema can never be blindly regenerated — it has to be **evolved**, one deliberate, reversible step at a time.

That's what **Alembic** (SQLAlchemy's migration tool) is for. A migration is a small Python file with exactly two functions: `upgrade()`, which describes how to move the schema one step forward, and `downgrade()`, which describes how to undo it. Alembic keeps a table inside Postgres itself that records which migrations have already run, so it always knows the database's exact current version and exactly which migrations still need to apply.

💡 CONCEPT

Autogenerate isn't magic, it's a diff. Running `alembic revision --autogenerate` connects to the live database, inspects its actual current tables, compares that against what the SQLAlchemy models *say* the schema should look like, and writes a migration file containing only the difference. It's closer to `git diff` than to a code generator — and just like a diff, you're expected to read what it produced before trusting it.

Here's the migration this project's models actually produced, autogenerated the moment all four tables existed only as Python classes and nothing yet existed in Postgres:

```
def upgrade() → None:
    op.create_table('categories',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('name', sa.String(), nullable=False),
        sa.PrimaryKeyConstraint('id')
    )
    op.create_index(op.f('ix_categories_name'), 'categories', ['name'], unique=True)

    op.create_table('users',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('email', sa.String(), nullable=False),
        sa.Column('hashed_password', sa.String(), nullable=False),
        sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'),
        nullable=False),
        sa.PrimaryKeyConstraint('id')
    )
    op.create_index(op.f('ix_users_email'), 'users', ['email'], unique=True)

    op.create_table('expenses',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('user_id', sa.Integer(), nullable=False),
        sa.Column('category_id', sa.Integer(), nullable=False),
        sa.Column('amount', sa.Numeric(precision=10, scale=2), nullable=False),
        sa.Column('description', sa.Text(), nullable=True),
        sa.Column('date', sa.Date(), nullable=False),
        sa.ForeignKeyConstraint(['category_id'], ['categories.id'], ),
        sa.ForeignKeyConstraint(['user_id'], ['users.id'], ),
        sa.PrimaryKeyConstraint('id')
    )
    # ### budgets follows the same shape as expenses, omitted here for space ###
```

Read it as literally as it's written: it's just Python calling `op.create_table`, `op.create_index`, and friends — the same operations you'd type into `psql` by hand, just described declaratively enough that Alembic can also generate the *opposite* operations for `downgrade()`. Notice the table order isn't alphabetical: `categories` and `users` are created before `expenses`, because `expenses` has foreign keys pointing at both of them, and Postgres refuses to create a foreign key constraint against a table that doesn't exist yet. Alembic figured that ordering out from the dependency graph implied by our `ForeignKey(...)` declarations — we never told it the order explicitly.

Running this migration is a separate, explicit step from starting the app — in this project's Docker setup, it happens automatically on container start via `alembic upgrade head`, which walks forward from whatever version the database is currently at to the latest one, applying only what's missing. Run it again on an already-current database and it does nothing at all, which is exactly the safe, idempotent behavior you want from infrastructure tooling.



WAR STORY

Generating this exact migration is what first broke this build — not from a modeling mistake, but from a missing file. The full story, with the real traceback, is War Story #1 in Chapter 9. It's worth reading even (especially) though the fix turned out to be one file.

Configuration & Secrets

Every value that changes between your laptop and a production server — and everything that would be a disaster if it leaked — lives in exactly one place, and it isn't the source code.

The database URL for local development is

```
postgresql://postgres:postgres@localhost:5432/expense_tracker
```

In production it will be something entirely different, pointing at a managed database with a real password. If that string were typed directly into `database.py`, deploying to a new environment would mean editing source code — and worse, committing a real production password to Git, where it lives forever in history even if you delete it in a later commit.

The fix is a strict separation: code describes *behavior*, environment variables describe *this particular deployment*. This project uses **pydantic-settings** to bridge the two — a typed class where every field can be overridden by an environment variable of the same name, with a safe default for local development if none is set.

backend/app/config.py

PYTHON

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    database_url: str = "postgresql://postgres:postgres@localhost:5432/expense_tracker"
    secret_key: str = "change-me-in-production"
    algorithm: str = "HS256"
    access_token_expire_minutes: int = 30
    cors_origins: str = "http://localhost:5173"

settings = Settings()
```

The `.env` file this reads from is deliberately listed in `.gitignore` — it never gets committed, and each developer (and each deployment environment) keeps their own copy, seeded from a committed `.env.example`

that documents which variables exist without exposing any real values. This is standard practice, not a project-specific convention: the [Twelve-Factor App](#) methodology names "store config in the environment" as one of its core rules for exactly this reason.

`settings` is instantiated once, at import time, and every other file in the app imports that same object — `from app.config import settings`. There's no scattered `os.environ.get(...)` calls with mismatched default values drifting apart across files; there's one typed source of truth, and your editor will autocomplete `settings.secret_key` and flag a typo immediately.

⚠️ COMMON MISTAKE

Notice `secret_key` defaults to the obviously-fake `"change-me-in-production"`. That default exists purely so the app runs out of the box in development — it is not a placeholder you're expected to remember to change later under pressure. Treat any default secret as a loud, deliberate reminder, and make sure your deployment process fails loudly if a real one was never set, rather than quietly running with the fake one.

Data Validation with Pydantic

A database row and an API request look similar and are not the same thing. Confusing them is one of the most common security mistakes in backend code — this project keeps them permanently, structurally separate.

Look back at the `User` model from Chapter 3: it has an `id`, an `email`, a `hashed_password`, and a `created_at`. Now ask: when someone registers, what fields should the `POST /auth/register` request actually accept? Certainly not `hashed_password` — the client sends a plain-text password and the *server* decides how to hash it. Certainly not `id` or `created_at` — the database assigns those. If your API just accepted "any fields that look like a User," a client could set their own `id`, or worse, directly set a field never meant to be client-writable.

This project solves it with **Pydantic schemas** — classes that look a lot like SQLAlchemy models but describe the shape of an API request or response, not a database table. They live in a completely separate directory, `app/schemas/`, and every router picks exactly which schema is allowed in and which is allowed out.

backend/app/schemas/auth.py

PYTHON

```
from pydantic import BaseModel, EmailStr

class UserRegister(BaseModel):
    email: EmailStr
    password: str

class UserLogin(BaseModel):
    email: EmailStr
    password: str

class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"
```

`UserRegister` can *only* ever contain an email and a password — FastAPI rejects any request with a missing or wrongly-typed field before your endpoint function even runs, and `EmailStr` goes further, rejecting anything that isn't a syntactically valid email address. This is **validation at the boundary**: the one place in the whole app where untrusted input from the outside world gets checked, so that every line of code past that point can simply trust the data is well-formed.

The same split shows up for every resource, and it's worth noticing the asymmetry between "in" and "out" shapes:

backend/app/schemas/expense.py

PYTHON

```
from datetime import date
from decimal import Decimal

from pydantic import BaseModel

class ExpenseCreate(BaseModel):
    category_id: int
    amount: Decimal
    description: str | None = None
    date: date

class ExpenseRead(BaseModel):
    id: int
    category_id: int
    amount: Decimal
    description: str | None
    date: date

    model_config = {"from_attributes": True}
```

`ExpenseCreate` has no `id` — a client creating an expense doesn't get to choose its database ID, the database assigns it. `ExpenseRead` has one, because once it exists, the client needs it back to reference that expense later (to edit or delete it). And `model_config = {"from_attributes": True}` is the bridge back to Chapter 3: it tells Pydantic it's allowed to build an `ExpenseRead` directly from a SQLAlchemy `Expense` object's attributes, not just from a raw dictionary — which is what lets a router simply `return expense` and have FastAPI correctly serialize it to JSON.

CONCEPT

This is also what builds your API docs for free. Because every request and response shape is a typed Pydantic schema, FastAPI can introspect all of them and generate a complete, interactive reference at `/docs` automatically — accurate by construction, since it's reading the same schemas that actually validate every request, not a separately hand-written document that quietly goes stale.

Authentication & Security, For Real

Two questions sit underneath every "logged in" feature you've ever used: how do you check a password without ever storing it, and how does a stateless server remember who you are between requests? This chapter answers both, with the exact code that answers them here.

Never store a password. Store a hash.

If `users.hashed_password` stored the literal password, then anyone who ever gained read access to the database — an attacker, a careless backup, a bug in a debug endpoint — would have every user's real password in plain text. Because people reuse passwords across sites, that one breach becomes a breach of every other account those users hold. This is not a hypothetical: it is the single most common cause of large-scale credential leaks.

The fix is **hashing**: running the password through a one-way function that's cheap to compute forward and computationally infeasible to reverse. We use **bcrypt**, via the `passlib` library, which additionally *salts* every hash — mixing in random bytes unique to that password before hashing, so two users with the identical password "hunter2" get completely different stored hashes, defeating precomputed "rainbow table" lookups.

```
from datetime import datetime, timedelta, timezone

from jose import JWTError, jwt
from passlib.context import CryptContext

from app.config import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")


def hash_password(password: str) → str:
    return pwd_context.hash(password)


def verify_password(plain_password: str, hashed_password: str) → bool:
    return pwd_context.verify(plain_password, hashed_password)


def create_access_token(data: dict, expires_delta: timedelta | None = None) → str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (
        expires_delta or timedelta(minutes=settings.access_token_expire_minutes)
    )
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, settings.secret_key, algorithm=settings.algorithm)


def decode_access_token(token: str) → dict | None:
    try:
        return jwt.decode(token, settings.secret_key, algorithms=[settings.algorithm])
    except JWTError:
        return None
```

Notice `verify_password` never decrypts anything — decryption isn't possible, by design. It hashes the freshly-submitted password using the same salt recorded inside the stored hash, and compares the two hashes for an exact match. This is the entire trick: you can confirm a password is correct without the system ever needing to know what it was.

Proving identity on every request: JSON Web Tokens

Hashing solves login. It doesn't solve the next problem: Chapter 1 established that the backend is stateless — it keeps no memory of who logged in. So how does `GET /expenses`, thirty seconds after login, know which user is asking? The answer is a **JWT (JSON Web Token)**: a signed, self-contained proof of identity that the *client* holds onto and resubmits with every request, instead of the server holding onto a session.

A JWT is three Base64-encoded segments joined by dots: `header.payload.signature`. The payload (our code calls it `data`, and stuffs the user's ID into a field named `sub`, short for "subject" — a JWT convention) is *not* encrypted, just encoded — anyone can decode and read it, so it must never contain a secret. What makes it trustworthy isn't secrecy, it's the signature: the server signs the header and payload together using `settings.secret_key`, a value only the server knows. Change a single character of the payload and the signature no longer matches, so `jwt.decode( ... )` in `decode_access_token` raises `JWTError` and the token is rejected. The token also carries its own expiry (`exp`), so a stolen token isn't valid forever — in this project, 30 minutes by default, from `settings.access_token_expire_minutes`.

UNDER THE HOOD

Compare this to the older "session cookie" approach: there, the server generates a random session ID, stores *who that belongs to* in server-side memory or a shared cache, and the cookie is just that opaque ID. JWTs invert the trade-off — no server-side storage needed, which is exactly what makes them scale trivially across many server instances — at the cost of being harder to revoke early (the server can't simply "forget" a JWT the way it can delete a session record). Both are legitimate, widely-used designs; this project uses JWTs because the trade suits a stateless REST API.

The full round trip

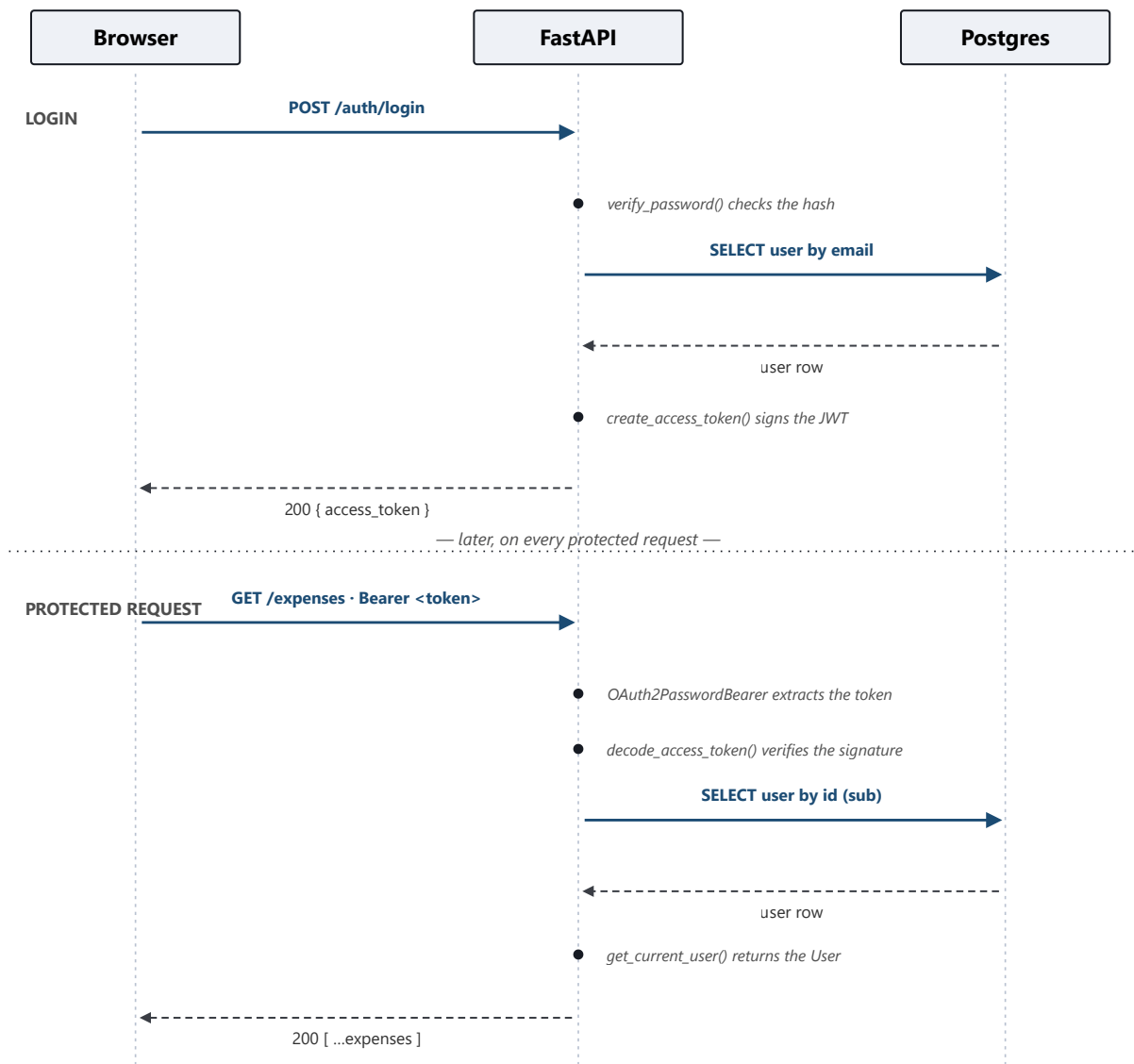


Fig. 3 — The same accent/dashed convention as Fig. 1: solid accent arrows carry a request forward, dashed arrows carry a response back. Notice login only touches the database once, but every single protected request touches it again to reload the user — a deliberate simplicity/performance trade this project makes, discussed below.

Making it a FastAPI convention: dependency injection

Every protected endpoint in this app needs the same three steps: pull the token out of the `Authorization` header, verify it, and load the corresponding user. Repeating that in every router function would be exactly the kind of duplicated, easy-to-get-subtly-wrong logic Chapter 2's separation of concerns is meant to prevent.

FastAPI's answer is **dependency injection**: you write the logic once as a function, and any route can ask FastAPI to run it first and hand over the result, just by declaring a parameter.

backend/app/dependencies.py

PYTHON

```
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session

from app.database import get_db
from app.models.user import User
from app.utils.security import decode_access_token

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")

def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
) → User:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )

    payload = decode_access_token(token)
    if payload is None:
        raise credentials_exception

    user_id = payload.get("sub")
    if user_id is None:
        raise credentials_exception

    user = db.get(User, int(user_id))
    if user is None:
        raise credentials_exception

    return user
```

`Depends( ... )` is the keyword to notice twice here. `Depends(oauth2_scheme)` tells FastAPI to look for an `Authorization: Bearer <token>` header and hand over just the token string (and, as a side effect, this is also what makes the interactive `/docs` page show a padlock icon and an "Authorize" button on every route that uses it). `Depends(get_db)` is the exact same `get_db` generator from Chapter 3 — dependencies can depend on other dependencies, and FastAPI resolves the whole chain automatically. Any route that wants "the currently logged-in user" simply writes `current_user: User = Depends(get_current_user)` as a parameter, and every check above — missing token, invalid signature, expired token, user deleted after the token was issued — is handled in one place, correctly, every time.

That reload from the database on every request (the second database hop in Fig. 3) is a deliberate trade: the JWT alone already proves *which user ID* is asking, but re-fetching the row means a user who's deleted their account mid-session is rejected immediately, rather than the token staying "valid" until it expires. For an app this size, one extra indexed primary-key lookup per request is a small price for that correctness.

Register, log in, and identify yourself: the router

With hashing, tokens, and the dependency in place, the actual auth endpoints are almost anticlimactic — which is exactly the point of building the hard parts as reusable pieces first.

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from app.database import get_db
from app.dependencies import get_current_user
from app.models.user import User
from app.schemas.auth import Token, UserLogin, UserRegister
from app.schemas.user import UserRead
from app.utils.security import create_access_token, hash_password, verify_password

router = APIRouter()

@router.post("/register", response_model=UserRead, status_code=status.HTTP_201_CREATED)
def register(payload: UserRegister, db: Session = Depends(get_db)):
    if db.query(User).filter(User.email == payload.email).first():
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Email already registered")

    user = User(email=payload.email, hashed_password=hash_password(payload.password))
    db.add(user)
    db.commit()
    db.refresh(user)
    return user

@router.post("/login", response_model=Token)
def login(payload: UserLogin, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == payload.email).first()
    if user is None or not verify_password(payload.password, user.hashed_password):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid email or password")

    access_token = create_access_token({"sub": str(user.id)})
    return Token(access_token=access_token)

@router.get("/me", response_model=UserRead)
def get_me(current_user: User = Depends(get_current_user)):
    return current_user
```

⚠️ COMMON MISTAKE

Look closely at the `login` function's error: it's identical — `"Invalid email or password"` — whether the email doesn't exist at all or the password was wrong. That's deliberate, not an oversight. A more "helpful" error like `"no account with that email"` lets an attacker silently enumerate which emails are registered, one guess at a time. Giving both failure cases the same generic message closes that door.

`/auth/me` wasn't in the original plan for this endpoint — it earns its place because the frontend needs a way to answer "who is currently logged in?" after a page refresh, when all it has is a token sitting in `localStorage` and no user object in memory. Chapter 10 shows exactly how it's used for that.

Building Real CRUD Endpoints

Authentication answers "who are you?" Authorization answers a different question: "are you allowed to touch this row?" Getting the second one wrong is how one user ends up reading another user's expenses.

CRUD — Create, Read, Update, Delete — is the pattern behind almost every resource an API exposes, and REST maps each operation onto an HTTP verb applied to a URL that names the resource, not the action: `POST /expenses/` to create, `GET /expenses/` to list, `PUT /expenses/{id}` to update, `DELETE /expenses/{id}` to remove. The URL is a noun; the verb is what changes.

VERB + PATH	MEANING	SUCCESS STATUS
<code>GET /expenses/</code>	list the caller's expenses	<code>200 OK</code>
<code>POST /expenses/</code>	create a new expense	<code>201 Created</code>
<code>PUT /expenses/{id}</code>	replace an existing expense	<code>200 OK</code>
<code>DELETE /expenses/{id}</code>	remove an expense	<code>204 No Content</code>

The status codes aren't decoration — they're part of the contract. `201` tells the client "a new thing now exists," distinct from `200`'s "here's the thing you asked for." `204` tells the client "this succeeded and there's deliberately no body to read," which is why `delete_expense` below has no `return` statement at all — returning `None` with a `204` status is correct, not accidental.

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from app.database import get_db
from app.dependencies import get_current_user
from app.models.category import Category
from app.models.expense import Expense
from app.models.user import User
from app.schemas.expense import ExpenseCreate, ExpenseRead

router = APIRouter()

def _get_owned_expense(db: Session, expense_id: int, current_user: User) → Expense:
    expense = db.get(Expense, expense_id)
    if expense is None or expense.user_id ≠ current_user.id:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Expense not found")
    return expense

def _validate_category(db: Session, category_id: int) → None:
    if db.get(Category, category_id) is None:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Category not found")

@router.get("/", response_model=list[ExpenseRead])
def list_expenses(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    return (
        db.query(Expense)
        .filter(Expense.user_id == current_user.id)
        .order_by(Expense.date.desc(), Expense.id.desc())
        .all()
    )

@router.post("/", response_model=ExpenseRead, status_code=status.HTTP_201_CREATED)
def create_expense(
    payload: ExpenseCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    _validate_category(db, payload.category_id)

    expense = Expense(user_id=current_user.id, **payload.model_dump())
    db.add(expense)
    db.commit()
```

```
db.refresh(expense)
return expense
```

Every function in this file takes `current_user: User = Depends(get_current_user)` — the dependency built in Chapter 7, reused rather than reinvented. And notice `list_expenses` doesn't query *all* expenses and filter client-side; the `.filter(Expense.user_id == current_user.id)` is baked directly into the SQL Postgres runs. This is the difference between **authentication** (Chapter 7: proving who you are) and **authorization** (this chapter: proving what you're allowed to see) — and authorization has to be enforced at the query itself, not as an afterthought applied to results that already left the database.

⚠ COMMON MISTAKE

`_get_owned_expense` is the single most important four lines in this file. A naive `update_expense` might load an expense purely by `expense_id` from the URL and update it — which would let any logged-in user edit or delete *any other user's* expense just by guessing or incrementing an ID in the URL. This class of bug is common enough to have a name: **Insecure Direct Object Reference**, and it's on the OWASP Top 10 for a reason. The fix is always the same shape: check ownership before acting, and if it fails, return `404` rather than `403` — so an attacker probing IDs can't even learn whether a given expense exists for someone else.

`categories.py` is the same pattern with less at stake: categories are shared, global defaults (Food, Transport, Books...) seeded once at startup by `seed_default_categories` from Chapter 2, not something users create themselves, so the endpoint is a straightforward authenticated read with no ownership filter needed:

backend/app/routers/categories.py

PYTHON

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.database import get_db
from app.dependencies import get_current_user
from app.models.category import Category
from app.models.user import User
from app.schemas.category import CategoryRead

router = APIRouter()

@router.get("/", response_model=list[CategoryRead])
def list_categories(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    return db.query(Category).order_by(Category.name).all()
```

It still requires `Depends(get_current_user)` even though it doesn't use `current_user` for anything — the goal isn't per-user filtering, it's simply refusing to answer an anonymous caller at all. A dependency doesn't have to change what a route *returns* to be worth declaring; sometimes its entire job is to gate access.

War Stories: Debugging Like an Engineer

Everything so far has been described as though it worked on the first try. It didn't. These are the three real failures hit while building exactly the code in this guide, and the reasoning that found each root cause.

This chapter is different from the rest: no new concepts, just the muscle that actually matters day to day — reading an error message closely enough to trust what it's telling you, instead of guessing. Every working engineer spends more time here than anywhere else.



WAR STORY #1 · THE MISSING TEMPLATE

Symptom. Running `alembic revision --autogenerate` (Chapter 4) to generate the very first migration failed with:

```
FileNotFoundError: [Errno 2] No such file or directory: 'alembic/script.py.mako'
```

Reading the traceback. Alembic doesn't write migration files from scratch — it fills in a *template* file to produce each new revision, the same way a project scaffolding tool fills in a boilerplate file. That template, `script.py.mako`, is normally created once by `alembic init` and is expected to sit alongside `env.py` in the `alembic/` directory permanently. The traceback's last frame was inside Mako (the templating library Alembic uses), trying to `open()` a file that simply wasn't there.

Root cause. The project's scaffold included `alembic/env.py` and an empty `alembic/versions/` folder, but the template file had never been created. Nothing about the Python code was wrong — the tooling itself was incomplete.

Fix. Write the standard Alembic template by hand: a Mako file whose placeholders (`${up_revision}` , `${message}` ...) Alembic fills in for every future migration.

Lesson. When a stack trace ends inside a library's own internals rather than your application code, the bug is rarely in that library — it's almost always a missing precondition your project was supposed to satisfy. Ask "what did this tool expect to already exist?" before assuming the tool itself is broken.

```

"""${message}

Revision ID: ${up_revision}
Revises: ${down_revision | comma,n}
Create Date: ${create_date}

"""
from alembic import op
import sqlalchemy as sa
${imports if imports else ""}

revision: str = ${repr(up_revision)}
down_revision = ${repr(down_revision)}

def upgrade() → None:
    ${upgrades if upgrades else "pass"}

```



WAR STORY #2 · THE RELATIONSHIP THAT COULDN'T FIND ITSELF

Symptom. With the migration fixed and the server starting for the first time, FastAPI crashed on boot:

```
sqlalchemy.exc.InvalidRequestError: When initializing mapper Mapper[User(users)], expression 'Budget' failed to locate a name ('Budget'). If this is a class name, consider adding this relationship() to the <class 'app.models.user.User'> class after both dependent classes have been defined.
```

Reading the traceback. Recall from Chapter 3 that `User.budgets = relationship("Budget", ...)` refers to the `Budget` class by *string name*, not a direct Python import — this is normal SQLAlchemy practice, since `user.py` and `budget.py` each import the other's class and a direct circular import would fail outright. But a string reference has to be resolved *eventually*, the first time any model is actually used, by searching every model class SQLAlchemy currently knows about. The error says plainly: it looked for a class named `Budget` and it wasn't registered anywhere yet.

Root cause. A Python class only gets registered with SQLAlchemy the moment its module is *imported* somewhere — merely existing as a file on disk isn't enough. `routers/auth.py` imports `User`. `routers/expenses.py` imports `Expense` and `Category`. But nothing in the whole application ever imported `budget.py` — the `/budgets` router doesn't exist yet (that's Part 2's work). So when FastAPI configured its mappers at startup and needed to resolve `"Budget"`, that class had simply never been loaded into memory.

Fix. Make `app/models/__init__.py` import every model explicitly, so importing the `models` package at all — regardless of which routers happen to be finished — guarantees every table class is registered:

backend/app/models/__init__.py

PYTHON

```
from app.models.budget import Budget
from app.models.category import Category
from app.models.expense import Expense
from app.models.user import User

__all__ = ["Budget", "Category", "Expense", "User"]
```

...and then, back in Chapter 2's `main.py`, one explicit line makes sure that package is imported before the app finishes starting up — `from app import models # noqa: F401`. That's the line that looked unused when it was first introduced. It wasn't.

CONCEPT

Lesson. "It's imported somewhere, so it must be loaded" is a Python-wide assumption worth distrusting. Import side effects (registering a class, running a decorator, populating a registry) only happen along the exact import paths your program actually executes. A class sitting in a file that nothing imports is, as far as the running program is concerned, a class that doesn't exist yet.

WAR STORY #3 · A DEPENDENCY THAT BROKE ITS OWN SELF-TEST

Symptom. With the server finally running, the very first `POST /auth/register` call — the first time `hash_password()` from Chapter 7 actually ran — returned a `500 Internal Server Error`. The server log ended in:

```
ValueError: password cannot be longer than 72 bytes, truncate manually if necessary (e.g.
my_password[:72])
```

Reading the traceback. The submitted password was eight characters — nowhere near 72 bytes. Reading further up the stack, the error wasn't happening while hashing *our* password at all: it originated inside `passlib`'s own internal `detect_wrap_bug()` routine, a one-time self-test `passlib` runs against the installed `bcrypt` library the first time it's used, to check for a historical bug in old `bcrypt` builds. *That* self-test hashes a fixed, longer diagnostic string — and it was *that* internal call raising the 72-byte error.

Root cause. `passlib`'s `bcrypt` self-test was written against the API `bcrypt` exposed in the 3.x/4.0 releases. This project's `requirements.txt` pinned `passlib[bcrypt] ≥ 1.7.4` with no upper bound on `bcrypt` itself, so pip installed the newest available `bcrypt` (5.x) — whose stricter input handling breaks that older self-test. This is a real, documented incompatibility between an unmaintained `passlib` release and modern `bcrypt`, not a mistake in this project's own code at all.

Fix. Pin the dependency that actually changed behavior: `bcrypt<4.1` alongside `passlib[bcrypt]`.

Lesson. Not every bug is yours to fix by rewriting code — some are a version-compatibility gap between two libraries that happen to be installed together, and the traceback tells you which one if you follow it past your own code and into the library frames.

backend/requirements.txt

TEXT

```
fastapi ≥ 0.115.0
uvicorn[standard] ≥ 0.32.0
sqlalchemy ≥ 2.0.36
psycopg2-binary ≥ 2.9.10
alembic ≥ 1.14.0
python-jose[cryptography] ≥ 3.3.0
passlib[bcrypt] ≥ 1.7.4
bcrypt < 4.1
python-dotenv ≥ 1.0.1
pydantic-settings ≥ 2.6.0
email-validator ≥ 2.2.0
```

Three bugs, three different layers of the stack — missing scaffolding, a Python import-ordering assumption, a dependency version clash — and not one of them was a typo in business logic. That's a representative sample of what debugging a real backend actually feels like, and it's why "read the whole traceback, including the parts inside libraries you didn't write" is worth more than any single tool.

Frontend Foundations

Part 2 builds the screens. This chapter builds the thin, boring, load-bearing layer underneath them — the plumbing that turns a JWT sitting in the browser into a React app that knows who's logged in.

A React component tree re-renders constantly and has no built-in memory between page loads. Two problems fall out of that immediately: every component that needs to call the API would otherwise have to know how to attach the JWT itself, and refreshing the browser would otherwise forget who's logged in entirely. This project solves both with two small, deliberately unglamorous files.

One Axios instance, one place that knows about tokens

Every API call in this app goes through a single configured `axios` instance rather than calling `fetch` directly from components. The payoff shows up immediately: an **interceptor**, a function Axios runs on every outgoing request before it's sent, is the one and only place that reads the token from storage and attaches it.

```
import axios from 'axios';
import { API_BASE_URL } from '../utils/constants';

const client = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

client.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default client;
```

Without this, every single file that calls the API — the expense form, the budget list, the analytics dashboard in Part 3 — would need to remember to read `localStorage` and set the header itself. Miss it once, in one file, and that one feature silently starts getting `401 Unauthorized` from the exact `get_current_user` dependency built in Chapter 7. Centralizing it here means that failure mode simply can't happen.

One source of truth for "who is logged in"

React's **Context API** lets a value be written once, high in the component tree, and read by any component below it without manually passing it down through every layer of props — exactly the tool for "is someone logged in," a fact nearly every screen in this app needs to know. `AuthContext` owns that fact, along with a subtlety a naive version would miss: on a fresh page load, the app only has a raw token string in storage, not a verified user — so it has to ask the server whether that token is still good before deciding anyone is really logged in.

```

export function AuthProvider({ children }) {
  const [token, setToken] = useState(() => localStorage.getItem('token'));
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    if (!token) {
      setIsLoading(false);
      return;
    }

    authApi
      .getMe()
      .then(setUser)
      .catch(() => {
        localStorage.removeItem('token');
        setToken(null);
        setUser(null);
      })
      .finally(() => setIsLoading(false));
  }, [token]);

  async function login(email, password) {
    const { access_token: accessToken } = await authApi.login(email, password);
    localStorage.setItem('token', accessToken);
    setToken(accessToken);
    const me = await authApi.getMe();
    setUser(me);
  }

  const value = {
    user,
    token,
    isAuthenticated: Boolean(token && user),
    isLoading,
    login,
    logout,
    register,
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

```

Trace the `useEffect`: it runs once on mount, and if a token exists, it calls exactly the `GET /auth/me` endpoint from Chapter 7 — the very reason that endpoint was added. If the request succeeds, `user` is populated and the session is restored transparently. If it fails (the token expired, or the secret key changed, or the user no longer exists), the stale token is discarded and the app falls back to logged-out, instead of trusting a token that *looks*

present but is no longer valid. Note also that `isAuthenticated` requires *both* a token *and* a loaded user — a token alone, before that verification round-trip finishes, is deliberately not enough.

💡 CONCEPT

Why does `isLoading` exist at all? Without it, a protected page would render for a single frame as "logged out" (because `user` starts as `null`) before the `/auth/me` check resolves — a visible flash-redirect to the login page on every refresh, even for someone who's legitimately logged in. `isLoading` lets the UI show a neutral loading state instead of jumping to a wrong conclusion early.

That's exactly what the route guard checks for. A component wrapping every protected page renders three ways depending on where `AuthContext` currently stands:

frontend/src/components/layout/ProtectedRoute.jsx

JSSX

```
export default function ProtectedRoute() {
  const { isAuthenticated, isLoading } = useAuth();

  if (isLoading) {
    return <LoadingSpinner />;
  }

  if (!isAuthenticated) {
    return <Navigate to={ROUTES.LOGIN} replace />;
  }

  return <Outlet />;
}
```

`<Outlet />` is React Router's placeholder for "whatever page was actually requested" — so this one small component is the single gate every protected page in the app passes through, and the only place "what happens if you're not logged in" has to be decided.

🔍 UNDER THE HOOD

Storing the token in `localStorage` is simple and works well for this project, but it's a real trade-off worth naming: unlike an `httpOnly` cookie, JavaScript running on the page can read it — including malicious JavaScript from a successful XSS attack. Production applications handling sensitive data often prefer `httpOnly` cookies for exactly this reason. It's a good example of how "the simple version" and "the most secure version" are genuinely different engineering decisions, not just more or less effort on the same one.

Proving It Works

A backend that "should work" and a backend you've watched work are different levels of confidence. Before building any UI on top of it, this API was proven end to end from the command line.

`curl` is a command-line tool for sending HTTP requests — the same requests a browser or a React app would send, just typed by hand. It's the fastest way to test a backend in isolation, before a frontend exists to blame for anything that goes wrong. The sequence below walks through the exact flow diagrammed in Fig. 3, one real terminal command at a time, each one depending on the previous succeeding first.

terminal

BASH

1. Register a new account

```
curl -X POST http://localhost:8000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"password123"}'
```

2. Log in and grab the JWT

```
curl -X POST http://localhost:8000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"password123"}'
```

3. Use the token to call a protected route

```
curl http://localhost:8000/auth/me \
  -H "Authorization: Bearer $TOKEN"
```

4. Create an expense

```
curl -X POST http://localhost:8000/expenses/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"category_id":1,"amount":12.50,"description":"Lunch","date":"2026-08-15"}'
```

Running these in order against the real, Dockerized stack (Chapter 2's `docker compose up`) is what actually caught War Story #3 in the first place: step 1 was the very first time `hash_password()` ran with a real request behind it, and it returned a `500` instead of a new user. That's the point of testing incrementally, piece by piece,

rather than building the entire stack and only then checking whether anything works — a failure at step 1 tells you immediately where to look, instead of leaving you to guess across five layers at once.

 CONCEPT

Why not just click around the UI? At this point in the build, the UI barely existed — and even once it does, a failed request through the UI tells you "something broke," while a failed `curl` command tells you exactly which endpoint, with exactly which payload, returned exactly which status code and body. Testing the API directly isn't a workaround for not having a frontend yet; professional backend work does this permanently, usually with tools like Postman or automated test suites that run the same idea continuously.

Each command also doubles as a live version of a concept from this guide: step 1 exercises the `UserRegister` schema and the uniqueness constraint from Chapter 6 and Chapter 3; step 2 exercises `verify_password` and `create_access_token` from Chapter 7; step 3 exercises the full `get_current_user` dependency chain; and step 4 exercises the ownership-scoped insert from Chapter 8. Four commands, and every major idea in this part of the guide has been proven against a running database, not just read about.

Recap & What's Next

Eleven chapters, four database tables, three real bugs, and one working, tested API. Here's everything introduced, in one table you can come back to.

CONCEPT	WHERE IT LIVES IN THIS PROJECT
Three-tier architecture	React (5173) · FastAPI (8000) · PostgreSQL (5432)
Separation of concerns	<code>models/</code> · <code>schemas/</code> · <code>routers/</code> · <code>utils/</code>
ORM & relationships	SQLAlchemy <code>Base</code> , <code>Column</code> , <code>relationship()</code> , <code>back_populates</code>
Schema migrations	Alembic <code>upgrade()</code> / <code>downgrade()</code> , autogenerate
Environment-based config	<code>pydantic-settings</code> , <code>.env</code> , <code>config.py</code>
Request/response validation	Pydantic schemas, separate from ORM models
Password hashing	<code>passlib</code> + bcrypt, salted, one-way
Stateless auth	JWTs signed with <code>python-jose</code> , verified per request
Dependency injection	FastAPI <code>Depends()</code> , <code>get_db</code> , <code>get_current_user</code>
REST conventions	Verbs, status codes, ownership-scoped CRUD
Global client state	React Context, Axios interceptors, protected routes

What exists right now, concretely: a user can register, log in, and receive a signed token; every expense and category endpoint is live, authenticated, and scoped correctly to its owner; the schema is version-controlled through a real migration; and the whole stack runs reproducibly through Docker. What doesn't exist yet is anything a user would call a *product* — there's no screen to actually add an expense, no budgets, and no charts. That's exactly the gap Part 2 closes.

Building the Product Surface

The Expense Form and Expense List get built against the API from Chapter 8, with real edit and delete actions. Budgets get their own CRUD endpoints and overspend logic — the "warn me before I go over" feature from the original user stories — plus the UI to set and see them. By the end of Part 2, this stops being an API you test with curl and becomes an app you'd actually use.

Glossary

Every term this guide introduced, in one place, for the moment three weeks from now when you half-remember one of them.

API

Application Programming Interface — here, the set of URLs FastAPI exposes for the frontend to call.

REST

An API style where URLs name resources and HTTP verbs (GET/POST/PUT/DELETE) name the action taken on them.

ORM

Object-Relational Mapper — a library (SQLAlchemy) that represents database rows as Python objects.

Primary key

The column that uniquely identifies a row within its table (`id` in every table here).

Foreign key

A column storing another table's primary key, the mechanism relationships are built from.

Migration

A version-controlled, reversible step that evolves a database schema without discarding data.

Pydantic schema

A typed class describing the shape of an API request or response, distinct from a database model.

Dependency injection

FastAPI's pattern for declaring shared request-time logic once (`Depends(...)`) and reusing it across routes.

Hashing

A one-way transformation of a password into a stored value that can be verified but never reversed.

Salt

Random data mixed into a password before hashing, so identical passwords produce different hashes.

JWT

JSON Web Token — a signed, self-contained proof of identity a stateless server can verify without storing sessions.

Authentication

Proving who is making a request.

Authorization

Proving what the authenticated requester is allowed to access or change.

CORS

Cross-Origin Resource Sharing — the browser policy that requires a server to explicitly allow requests from other origins.

Stateless

A server design where no per-user data is kept in server memory between requests.

Environment variable

A configuration value supplied outside the source code, so it can differ per deployment without editing code.